Technical Report: Exact k-NN Implementation

This document outlines the logic and structure of the Exact k-Nearest Neighbors (k-NN) algorithm implemented in C++.

It is divided into distinct parts: the foundational core that utilizes block-wise operations and OpenBLAS, and the extended parallelization structure utilizing Pthreads.

Part 1: Core Mathematical Foundation and Block-wise Operations

1. The Mathematical Foundation & Squared Norms

To avoid computationally expensive point-to-point Euclidean distance calculations using nested loops, the algorithm utilizes the expanded quadratic expansion form of Euclidean distance:

$$D^2 = ||C||^2 - 2CQ^T + ||Q||^2$$

The first step involves calculating the squared norms of the dataset vectors.

```
 void compute_squared_norms(const double* matrix, int rows, int cols,
vector<double>& norms) {
    for (int i = 0; i < rows; ++i) {
        double sum = 0.0;
        for (int j = 0; j < cols; ++j) {
            double val = matrix[i * cols + j];
            sum += val * val;
        }
        norms[i] = sum;
    }
}
```

Logic: This helper function iterates through the matrix row by row, squaring each element and summing them up to find the squared magnitude ($||C||^2$ and $||Q||^2$) of each vector. This allows the distance equation to piece together pre-computed components rather than iterating over every dimension repeatedly.

2. Block-wise Processing and OpenBLAS Integration

Processing massive matrices at once would lead to memory overflow. The algorithm solves this by breaking the query set Q into manageable blocks.

```
 int BLOCK_SIZE = 1000;
for (int b_start = 0; b_start < M; b_start += BLOCK_SIZE) {
    int current_B = min(BLOCK_SIZE, M - b_start);
    vector<double> D_block(current_B * N, 0.0);
    cblas_dgemm(CblasRowMajor, CblasNoTrans, CblasTrans,
                current_B, N, d,
                -2.0, Q + b_start * d, d, C, d,
                0.0, D_block.data(), N);
```

Logic: The outer loop slices the queries into chunks of 1,000. For the heaviest computational task—the dot product $-2CQ^T$—the code calls `cblas_dgemm` from the OpenBLAS library. This performs a highly optimized Matrix-Matrix multiplication, entirely bypassing the need for slow manual nested loops.

3. Distance Assembly and Fast Sorting (Quick-Select)

After calculating the complex matrix multiplication, the final step is to assemble the actual distance values and extract the k nearest neighbors.

```
     for (int j = 0; j < N; ++j) {
        double dist_sq = Q_block_sq[i] + C_sq[j] + D_block[i * N + j];
        dist_sq = max(0.0, dist_sq);
        row_dists[j] = {sqrt(dist_sq), j};
    }
    // Quick-select
    nth_element(row_dists.begin(), row_dists.begin() + k, row_dists.end());
    sort(row_dists.begin(), row_dists.begin() + k);
}
```

Logic: The pre-calculated squared norms and the OpenBLAS dot product are added together. The `max(0.0, ...)` safeguard handles microscopic floating-point precision errors that might result in negative numbers before taking the square root. Once the full row of distances is assembled, `std::nth_element` is used. This is a crucial optimization: instead of sorting all N distances (which takes O(N log N) time), `nth_element` partially sorts the array to find the top k elements in O(N) time. A final small sort organizes just those k items.

Part 2: Parallelization with Pthreads

4. Parallelization Strategy: Pthreads and OpenBLAS Synergy

To further accelerate the Exact k-NN implementation, the workload is distributed across multiple CPU cores using POSIX Threads (pthreads). Because the algorithm evaluates distinct blocks of the query matrix (Q) independently against the corpus matrix (C), the problem is "embarrassingly parallel".

```
openblas_set_num_threads(1);

int num_threads = sysconf(_SC_NPROCESSORS_ONLN); // πυρήνων
if (num_threads <= 0) num_threads = 4;

int chunk_size = M / num_threads;
```

Logic: The critical first step is calling `openblas_set_num_threads(1)`. OpenBLAS natively tries to multithread matrix operations. If we spawn our own pthreads and OpenBLAS also spawns threads inside each of those, it leads to "oversubscription" (creating vastly more threads than physical CPU cores), which causes severe context-switching overhead and degrades performance. By locking OpenBLAS to a single thread per call, we allow our outer pthreads structure to cleanly handle the parallelization.

```
for (int i = 0; i < num_threads; ++i) {
    thread_data[i] = {C, Q, N, M, d, k,
                      i * chunk_size,
                      (i == num_threads - 1) ? M : (i + 1) * chunk_size,
                      &C_sq, &idx, &dst};
    pthread_create(&threads[i], NULL, exact_knn_worker, &thread_data[i]);
}
```

Logic: The total number of query rows (M) is divided evenly by the number of logical cores (`chunk_size`). The `ExactThreadData` structure acts as a payload, passing the memory pointers and specifically defining the `start_row` and `end_row` for each thread. Because each thread writes to entirely distinct, pre-allocated sections of the final `idx` and `dst` vectors, there is no risk of race conditions, and therefore, no need for slow mutex locks during the distance calculation.

Technical Report: Producer-Consumer

Task Queue Refactoring for Generic Execution

To transition the system from a simple integer-passing demonstration to a robust, dynamic task dispatcher, the foundational data structures were modified. The queue now holds structured function pointers rather than primitive types.

```
 struct workFunction {
   void * (*work)(void *);
   void * arg;
};

typedef struct {
   struct workFunction buf[QUEUESIZE];
   long head, tail;
   int full, empty;
   pthread_mutex_t *mut;
   pthread_cond_t *notFull, *notEmpty;
} queue;
```

Logic: By substituting the integer array with an array of `workFunction` structs, the queue becomes completely agnostic to the type of work being processed. This is a common architectural pattern in embedded systems and DSP software, where a centralized thread pool must handle various incoming asynchronous signal processing tasks. The producer is now responsible for packaging the specific function (`calculate_sine`) along with its dynamically allocated arguments into a single structural unit before pushing it to the FIFO queue.

Continuous Consumption and Artificial Delay Removal

The behavior of both the producer and consumer threads was overhauled to reflect a high-performance, continuous processing environment, eliminating the restrictive `sleep()` functions and bounded consumer loops.

```
void *consumer (void *q) {
  // ... [queue setup and locking] ...
  while (1) {
    // ... [condition waits and extraction] ...
    queueDel (fifo, &task);
    pthread_mutex_unlock (fifo->mut);
    pthread_cond_signal (fifo->notFull);

    task.work(task.arg); // Task executed outside the critical section
```

```
  }
}
```

Logic: The original code artificially constrained execution using `usleep()` and a double-loop structure. These were stripped out entirely. The consumer was converted into a continuous background worker using a `while(1)` loop, effectively acting as an always-ready thread pool. Crucially, the actual execution of the payload (`task.work(task.arg)`) occurs *after* the `pthread_mutex_unlock`. This ensures that the heavy computational work—such as calculating the series of sine waves—happens concurrently across multiple cores without unnecessarily holding the mutex lock and stalling the queue for other threads.

Memory Management and Workload Distribution

To simulate a realistic workload without memory leaks, the producer and the dispatched function must work in tandem regarding memory ownership.

```
 // Inside Producer:
double *arg = (double *)malloc(sizeof(double));
*arg = (double)i * 0.05;
task.arg = arg;

// Inside calculate_sine:
// ... [sine calculations] ...
free(arg);
```

Logic: Because the producer operates in a fast, tight loop (`LOOP = 5000`), passing arguments by reference to local stack variables would lead to race conditions and corrupted data by the time the consumer executes them. Therefore, the producer dynamically allocates memory for each argument on the heap. Ownership of this memory is conceptually transferred through the queue to the consumer. Once the `calculate_sine` function completes its DSP computations, it takes responsibility for freeing the memory, ensuring stable operation over millions of cycles without memory exhaustion.

Nikolaos Karapatsias 10661

Code: https://github.com/nkarapatsias/Real-Time-Embedded-Systems